Introduction to Analysis of Algorithms

What is an algorithm?

Algorithm is a *set of steps to complete a task.*

For example,

Task: to make a cup of tea.

Algorithm:

- add water and milk to the kettle,
- boil it, add tea leaves,
- Add sugar, and then serve it in cup.

What is *Computer algorithm*?

''a set of steps to accomplish or complete a task that is described precisely enough that a computer can run it''.

Described precisely: very difficult for a machine to know how much water, milk to be added etc. in the above tea making algorithm.

These algorithms run on computers or computational devices. For example, GPS in our smartphones, Google hangouts.

GPS uses *shortest path algorithm.* Online shopping uses cryptography which uses RSA algorithm.

Characteristics of an algorithm: -

- Must take an input.
- Must give some output (yes/no, value etc.)
- Definiteness –each instruction is clear and unambiguous.
- Finiteness –algorithm terminates after a finite number of steps.
- Effectiveness –every instruction must be basic i.e. simple instruction.

Expectation from an algorithm

- Correctness:-
  - Correct: Algorithms must produce correct result.
  - Produce an incorrect answer: Even if it fails to give correct results all the time still there is a control on how often it gives wrong result. Eg. Rabin-Miller Primality Test (Used in RSA algorithm): It doesn't give correct answer all the time.1 out of 2^{50} times it gives incorrect result.
  - Approximation algorithm: Exact solution is not found, but near optimal solution can be found out. (Applied to optimization problem.)
- Less resource usage:

  Algorithms should use less resources (time and space).

Resource usage:

Here, the time is considered to be the primary measure of efficiency. We are also concerned with how much the respective algorithm involves the computer memory. But mostly time is the resource that is dealt with. And the actual running time depends on a variety of backgrounds: like the speed of the Computer, the language in which the algorithm is implemented, the compiler/interpreter, skill of the programmers etc.

So, mainly the resource usage can be divided into: 1. Memory (space) 2 . Time

Time taken by an algorithm?

- performance measurement or Apostoriori Analysis: Implementing the algorithm in a machine and then calculating the time taken by the system to execute the program successfully.
- Performance Evaluation or Apriori Analysis. Before implementing the algorithm in a system. This is done as follows

1. How long the algorithm takes:- will be represented as a function of the size of the input.

f(n)→how long it takes if 'n' is the size of input.

2. How fast the function that characterizes the running time grows with the input size.

   "Rate of growth of running time".

   The algorithm with less rate of growth of running time is considered better.

How algorithm is a technology?

Algorithms are just like a technology. We all use latest and greatest processors but we need to run implementations of good algorithms on that computer in order to properly take benefits of our money that we spent to have the latest processor. Let's make this example more concrete by pitting a faster computer (computer A) running a sorting algorithm whose running time on n values grows like n^2 against a slower computer (computer B) running a sorting algorithm whose running time grows like n lg n. They each must sort an array of 10 million numbers. Suppose that computer A executes 10 billion instructions per second (faster than any single sequential computer at the time of this writing) and computer B executes only 10 million instructions per second, so that computer A is1000 times faster than computer B in raw computing power. To make the difference even more dramatic, suppose that the world's craftiest programmer codes in machine language for computer A, and the resulting code requires $2n^2$ instructions to sort n numbers. Suppose further that just an average programmer writes for computer B, using a high- level language with an inefficient compiler, with the resulting code taking 50n lg n instructions.

Computer A (Faster)	Computer B(Slower)
Running time grows like n^2.	Grows in nlogn.
10 billion instructions per sec.	10 million instruction per sec
$2n^2$ instruction.	50 nlogn instruction.

Time taken= $\frac{2 \times (10^7)^2}{10^{10}} = 20{,}000$ $\frac{50 \times 10^7 \times \log 10^7}{10^7} \approx 1163$

It is more than 5.5hrs it is under 20 mins.

So choosing a good algorithm (algorithm with slower rate of growth) as used by computer B affects a lot.

Growth of Functions (Asymptotic notations)

Before going for growth of functions and asymptotic notation let us see how to analyze an algorithm.

How to analyse an Algorithm

Let us form an algorithm for Insertion sort (which sort a sequence of numbers).The pseudo code for the algorithm is give below.

Pseudo code:

for j=2 to A length ---------- C_1

key=A[j]----------C_2

//Insert A[j] into sorted Array A[1.....j-1]---------- C_3

i=j-1----------C_4

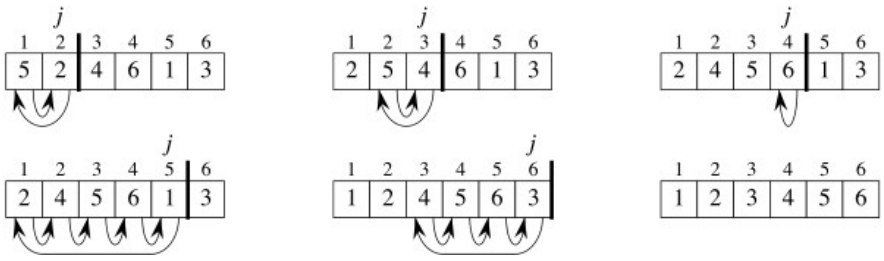
while i>0 & A[j]>key----------C_5

A[i+1]=A[i]----------C_6

i=i-1----------C_7

A[i+1]=key----------C_8

Example:



Let C_i be the cost of ith line. Since comment lines will not incur any cost $C_3=0$.

Cost	No. Of times Executed
C_1	n
C_2	n-1
$C_3=0$	n-1
C_4	n-1
C_5	$\sum_{j=2}^{n-1} t_j$
C_6	$\sum_{j=2}^{n}(t_j - 1)$
C_7	$\sum_{j=2}^{n}(t_j - 1)$
C_8	n-1

Running time of the algorithm is:

$T(n)=C_1n+C_2(n-1)+0(n-1)+C_4(n-1)+C_5(\sum_{j=2}^{n-1} t_j)+C_6(\sum_{j=2}^{n} t_j - 1)+C_7(\sum_{j=2}^{n} t_j - 1)+ C_8(n-1)$

Best case:

It occurs when Array is sorted.

All t_j values are 1.

$T(n)=C_1n+C_2(n-1)+0\ (n-1)+C_4(n-1)+C_5(\sum_{j=2}^{n-1} 1)+C_6(\sum_{j=2}^{n} 0)+C_7(\sum_{j=2}^{n} 0)+\ C_8(n-1)$

$=C_1n+C_2\ (n-1)+0\ (n-1)+C_4\ (n-1)+C_5(n-1)+\ C_8\ (n-1)$

$= (C_1+C_2+C_4+C_5+\ C_8)\ n-(C_2+C_4+C_5+\ C_8)$

- Which is of the form an+b.
- Linear function of n. So, linear growth.

Worst case:

It occurs when Array is reverse sorted, and t_j =j

$T(n)=C_1n+C_2(n-1)+0\ (n-1)+C_4(n-1)+C_5(\sum_{j=2}^{n-1} j)+C_6(\sum_{j=2}^{n} j-1)+C_7(\sum_{j=2}^{n} j-1)+\ C_8(n-1)$

$=C_1n+C_2(n-1)+C_4(n-1)+C_5\ (\frac{n(n-1)}{2}-1)+C_6(\sum_{j=2}^{n}\frac{n(n-1)}{2})+C_7(\sum_{j=2}^{n}\frac{n(n-1)}{2})+\ C_8(n-1)$

which is of the form an^2+bn+c

Quadratic function. So in worst case insertion set grows in n^2.

Why we concentrate on worst-case running time?

- The worst-case running time gives a guaranteed upper bound on the running time for any input.
- For some algorithms, the worst case occurs often. For example, when searching, the worst case often occurs when the item being searched for is not present, and searches for absent items may be frequent.
- Why not analyze the average case? Because it's often about as bad as the worst case.

Order of growth:

It is described by the highest degree term of the formula for running time. (Drop lower-order terms. Ignore the constant coefficient in the leading term.)

Example: We found out that for insertion sort the worst-case running time is of the form $an^2 + bn + c$.

Drop lower-order terms. What remains is an^2.Ignore constant coefficient. It results in n^2. But we cannot say that the worst-case running time T(n) equals n^2 .Rather It grows like n^2 . But it doesn't equal n^2. We say that the running time is $\Theta(n^2)$ to capture the notion that the order of growth is n^2.

We usually consider one algorithm to be more efficient than another if its worst-case running time has a smaller order of growth.

Asymptotic notation

- It is a way to describe the characteristics of a function in the limit.
- It describes the rate of growth of functions.
- Focus on what's important by abstracting away low-order terms and constant factors.
- It is a way to compare "sizes" of functions:

  O≈ ≤

  Ω≈ ≥

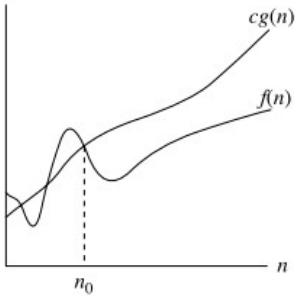
  Θ ≈ =

  o ≈ <

  ω ≈ >

O-notation

$O(g(n)) = \{f(n) :$ there exist positive constants c and n_0 such that
$0 \le f(n) \le cg(n)$ for all $n \ge n_0\}$.



$g(n)$ is an ***asymptotic upper bound*** for $f(n)$.

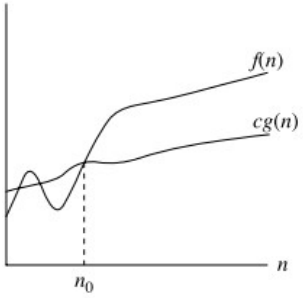
Example: $2n^2 = O(n^3)$, with $c = 1$ and $n_0 = 2$.

Examples of functions in $O(n^2)$:

n^2
$n^2 + n$
$n^2 + 1000n$
$1000n^2 + 1000n$
Also,
n
$n/1000$
$n^{1.99999}$
$n^2/\lg\lg\lg n$

Ω-notation

$\Omega(g(n)) = \{f(n) :$ there exist positive constants c and n_0 such that $0 \le cg(n) \le f(n)$ for all $n \ge n_0\}$.



$g(n)$ is an ***asymptotic lower bound*** for $f(n)$.

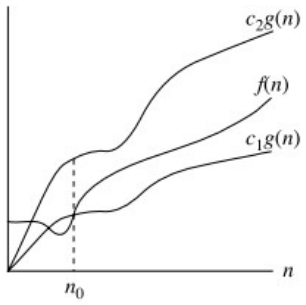
Example: $\sqrt{n} = \Omega(\lg n)$, with $c = 1$ and $n_0 = 16$.

Examples of functions in $\Omega(n^2)$:

n^2
$n^2 + n$
$n^2 - n$
$1000n^2 + 1000n$
$1000n^2 - 1000n$
Also,
n^3
$n^{2.00001}$
$n^2 \lg\lg\lg n$
2^{2^n}

Θ-notation

$\Theta(g(n)) = \{f(n) :$ there exist positive constants c_1, c_2, and n_0 such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0\}$.



$g(n)$ is an ***asymptotically tight bound*** for $f(n)$.

Example: $n^2/2 - 2n = \Theta(n^2)$, with $c_1 = 1/4$, $c_2 = 1/2$, and $n_0 = 8$.

***o*-notation**

$o(g(n)) = \{f(n) :$ for all constants $c > 0$, there exists a constant $n_0 > 0$ such that $0 \le f(n) < cg(n)$ for all $n \ge n_0\}$.

Another view, probably easier to use: $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.

$n^{1.9999} = o(n^2)$
$n^2/\lg n = o(n^2)$
$n^2 \ne o(n^2)$ (just like $2 \not< 2$)
$n^2/1000 \ne o(n^2)$

***ω*-notation**

$\omega(g(n)) = \{f(n) :$ for all constants $c > 0$, there exists a constant $n_0 > 0$ such that $0 \le cg(n) < f(n)$ for all $n \ge n_0\}$.

Another view, again, probably easier to use: $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$.

$n^{2.0001} = \omega(n^2)$
$n^2 \lg n = \omega(n^2)$
$n^2 \ne \omega(n^2)$

In asymptotic notation, *little o* (written as $f(n) = o(g(n))$) and *little omega* (written as $f(n) = \omega(g(n))$) describe how a function grows relative to another function as n (often representing input size) becomes very large.

Here's a simplified breakdown:

1. **Little o** ($f(n) = o(g(n))$): This means that $f(n)$ grows slower than $g(n)$ as $n \to \infty$. Specifically, $f(n)$ becomes insignificant compared to $g(n)$ for large n. Mathematically, we say $f(n) = o(g(n))$ if:

   $$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

   In other words, $f(n)$ grows at a rate slower than $g(n)$ and eventually becomes negligible in comparison.

   Example: If $f(n) = n$ and $g(n) = n^2$, then $f(n) = o(g(n))$ because n grows much slower than n^2 as n gets large.

2. **Little omega** ($f(n) = \omega(g(n))$): This means that $f(n)$ grows faster than $g(n)$ as $n \to \infty$. Mathematically, we say $f(n) = \omega(g(n))$ if:

   $$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$$

   This shows that $f(n)$ eventually outpaces $g(n)$ as n increases.

   Example: If $f(n) = n^2$ and $g(n) = n$, then $f(n) = \omega(g(n))$ because n^2 grows much faster than n for large values of n.

In summary:

- **Little o** notation shows a function grows slower than another.
- **Little omega** notation shows a function grows faster than another.